Multi-Object Synchronization

Description

Large Programs

When synchronizing across multiple objects in large programs:

- Each object has its own lock and condition variables
- The modularity of locking is a concern
- Performance implications arise
- Deadlock risks increase
- Other crucial aspects include:
  - Semantics and correctness
  - Eliminating locks when possible

Synchronization Performance

Even with many threads, performance can degrade on a multiprocessor system due to:

- Thread creation overhead (if unnecessary)
- Lock contention
- High cache-coherence memory traffic
- **False sharing**: unnecessary communication between cores for data that isn't truly shared but resides in the same cache block

Definitions

- **Resource**: Any passive entity needed by a thread (e.g., CPU, disk space, memory, lock)
  - ***Preemptable***: Can be taken away by the OS and later restored
  - ***Non-preemptable***: Must remain with the thread

- **Starvation**: A thread waits indefinitely
  - **Starvation ≠ Deadlock**

Semaphore

A semaphore is a variable used to control access to a shared resource. It can take non-negative integer values and supports two atomic operations:

- `P()` or `wait()` (also known as down or decrement):
  - If the semaphore value is greater than zero, it is decremented.
  - If it is zero, the process is blocked until the semaphore becomes positive.
- `V()` or `signal()` (also known as up or increment):
  - Increments the semaphore value.
  - If there are processes waiting, it unblocks one of them.

Formal Representation

A semaphore `S` is characterized by:

- An integer value `S.value`
- A queue `S.queue` for waiting processes

The atomic operations are defined as:

```
void wait(S) {
    while (S.value == 0) ;  // Busy waiting
    S.value--;
}

void signal(S) {
    S.value++;
}
```

However, this implementation is inefficient in preemptive environments. In real systems, `wait()` puts the process to sleep instead of using CPU cycles with busy waiting.

Types of Semaphores

1. **Binary Semaphores (or Mutexes)**
   - Can only take values 0 or 1.
   - When initialized to 1, it is used for mutual exclusion between processes.

   Example usage:

   ```
   semaphore mutex = 1;  // Initially available

   void process() {
       wait(mutex);  // Acquire semaphore
       // Critical section
       signal(mutex); // Release semaphore
   }
   ```

   If two processes execute `wait(mutex)`, only one will enter the critical section.

2. **Counting Semaphores**
   - Can take values greater than 1.
   - Used for resources with multiple instances available.
   - If initialized to `N`, up to `N` processes can access it simultaneously.

   Example: Suppose there are 3 printers available, we can use a semaphore initialized to 3.

   ```
   semaphore printers = 3;

   void print_document() {
       wait(printers);  // Acquire a printer
       // Print the document
       signal(printers); // Release the printer
   }
   ```

If 3 processes are printing, the fourth must wait.

3. **Generalized Semaphores**
   - Used for advanced synchronization.
   - Can take any integer value.
   - Used in barriers and thread synchronization.

3. Applications of Semaphores

1. **Mutual Exclusion (Mutex)**
   - Binary semaphores guarantee that only one process at a time can access a resource.

```
semaphore mutex = 1;

void critical_section() {
    wait(mutex);  // Enter critical section
    // Critical section
    signal(mutex); // Exit critical section
}
```

2. **Producer-Consumer (Circular Buffer)**
   - A classic synchronization problem. It uses:
     - A semaphore to count items in the buffer.
     - A semaphore for available space.
     - A mutex to execute operations with mutual exclusion.

```
semaphore full = 0;   // Number of items in the buffer
semaphore empty = N;  // Available space in the buffer
semaphore mutex = 1;  // Controls access to the buffer

void producer() {
    int item;
```

```
    while (true) {
        item = produce_item();
        wait(empty);  // Wait for available space
        wait(mutex);  // Acquire exclusive access
        insert_item(item);
        signal(mutex); // Release buffer
        signal(full);  // Increase number of items
    }
}

void consumer() {
    int item;
    while (true) {
        wait(full);   // Wait for an item in the buffer
        wait(mutex);  // Acquire exclusive access
        item = remove_item();
        signal(mutex); // Release buffer
        signal(empty); // Increase available space
        consume_item(item);
    }
}
```

How it works:

- The producer creates an item and puts it into the buffer.
- If the buffer is full, it waits (`wait(empty)`).
- The consumer takes an item from the buffer.
- If the buffer is empty, it waits (`wait(full)`).

3. **Readers and Writers Problem**
    - Readers can access the resource concurrently.
    - Writers must have exclusive access.

```
semaphore mutex = 1;
semaphore wrt = 1;
```

```
int read_count = 0;

void reader() {
    wait(mutex);
    read_count++;
    if (read_count == 1) wait(wrt); // First reader blocks
writers
    signal(mutex);

    // Reading

    wait(mutex);
    read_count--;
    if (read_count == 0) signal(wrt); // Last reader unblo
cks writers
    signal(mutex);
}

void writer() {
    wait(wrt);  // Exclusive access
    // Writing
    signal(wrt);
}
```

4. **Dining Philosophers Problem**
   - Philosophers share chopsticks and must avoid deadlock.

```
semaphore chopstick[5];

void philosopher(int i) {
    while (true) {
        think();
        wait(chopstick[i]);          // Take the left chop
stick
        wait(chopstick[(i+1)%5]);    // Take the right cho
```

```
pstick
        eat();
        signal(chopstick[i]);        // Release the left c
hopstick
        signal(chopstick[(i+1)%5]);  // Release the right
chopstick
    }
}
```

Possible solution to deadlock:

- A philosopher first picks up the right chopstick, then the left, except the last one who does the opposite.

Deadlock Definition

A **set of processes is in a deadlock state** when each process in the set is waiting for an event that only another process in the set can trigger.

- ***The key factor***: all processes are stuck in a waiting state
- Deadlock implies starvation, but starvation does not necessarily imply deadlock

Assumptions

We consider **reusable resources**, such as critical sections, printers, file descriptors.

- **Request Phase**: `lock.acquire()`, `cond.wait(&lock)` (assumption: blocking request)
- **Assignment and Use**: If a thread must wait, it retains the resource
- **Release Phase**: `lock.release()`, `cond.signal()`

Code Example: Two Locks

***Thread A*:**

```
lock1.acquire()
lock2.acquire()
lock2.release()
lock1.release()
```

Thread B:

```
lock2.acquire()
lock1.acquire()
lock1.release()
lock2.release()
```

- **Potential Deadlock Scenario**:
  If *Thread A* acquires `lock1` while *Thread B* acquires `lock2`, both will indefinitely wait for the other lock.

Code Example: Bidirectional Bounded Buffer

Thread A:

```
buffer1.put(data)
buffer1.put(data)
buffer2.get()
buffer2.get()
```

Thread B:

```
buffer2.put(data)
buffer2.put(data)
buffer1.get()
buffer1.get()
```

- **Deadlock Risk**:
  If both buffers have only one empty slot left, Thread A and Thread B can get

stuck waiting for space to free up.

Dining Philosophers

The "Classic Problem" of the Dining Philosophers is a well-known synchronization problem in computer science, used to illustrate concurrency concepts and issues related to managing shared resources. Let's break down the problem and the provided code:

Problem Context:

- Imagine five philosophers sitting around a table. Each philosopher alternates between thinking and eating.
- To eat, each philosopher needs two chopsticks: one on their left and one on their right.
- Each philosopher can pick up one chopstick at a time. The order in which they pick up the chopsticks is crucial to avoid deadlock (a situation where no progress is made).

How the Code Works:

The provided code describes the actions of a single philosopher (denoted as `id`).

```
void philosopher (int id)
while(TRUE) {
    think_for_a_while(); // Philosopher thinks for a while
    take_chopstick(id); // Takes the left chopstick
    take_chopstick((id+1) % 5); // Takes the right chopstick
    eat(); // Eats
    put_fork(id); // Puts down the left chopstick
    put_fork((id+1)%5); // Puts down the right chopstick
}
```

Flow Explanation:

1. **Thinking** (`think_for_a_while()`)
   Each philosopher spends part of their time thinking.

2. **Taking the Left Chopstick** (`take_chopstick(id)`)
   The philosopher picks up the chopstick to their left.
3. **Taking the Right Chopstick** (`take_chopstick((id+1)%5)`)
   Then, they pick up the chopstick to their right.
4. **Eating** (`eat()`)
   After both chopsticks are taken, the philosopher can eat.
5. **Putting Down the Left Chopstick** (`put_fork(id)`)
   Once finished, the philosopher puts down the left chopstick.
6. **Putting Down the Right Chopstick** (`put_fork((id+1)%5)`)
   Finally, the philosopher puts down the right chopstick.

Deadlock Problem:

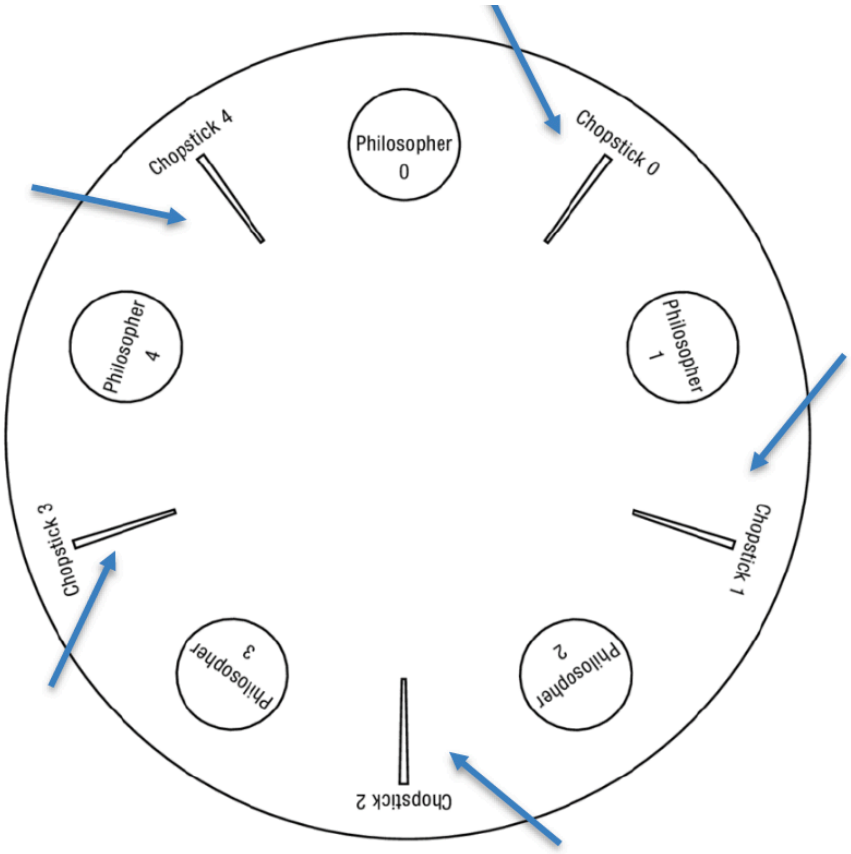
The described code can easily lead to **deadlock**. Here's why:

1. Each philosopher first picks up the left chopstick, so all philosophers could end up holding their left chopstick (because each philosopher picks their left chopstick before the right one).
2. If all philosophers have their left chopstick, none will be able to pick up the right chopstick (since each right chopstick is already held by the philosopher on the left).
3. This results in a situation where all philosophers are stuck, each waiting for the right chopstick, but none can get it. This is known as **deadlock**.

Solutions:

To avoid deadlock, various solutions have been proposed, such as:

- **Hierarchy in chopstick picking**: Philosophers pick the right chopstick first, or establish a fixed order for acquiring the chopsticks to prevent everyone from simultaneously picking up the left chopstick.
- **Monitors and Semaphores**: Use synchronization mechanisms to control access to shared resources (the chopsticks), ensuring that at least one philosopher can eat at any given time.

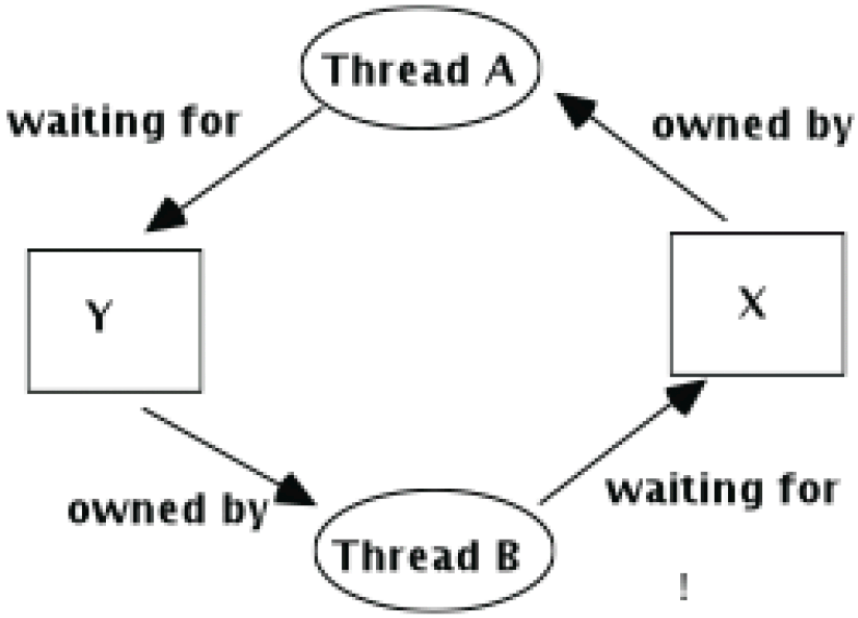


Example Solution: Arbitrator Approach

- Introduce a waiter who grants permission to pick up both chopsticks before a philosopher can start eating.
- This avoids circular waiting by enforcing an external order of access.

Necessary Conditions for Deadlock

1. **Mutual Exclusion:** A resource can only be held by one thread at a time.
2. **No Preemption:** Resources cannot be forcibly taken away.
3. **Wait While Holding:** A thread holds resources while waiting for others.
4. **Circular Waiting:** A cycle of dependencies exists in resource requests.
    - If there is only **one instance of each resource**, circular waiting is both a necessary and **sufficient** condition for deadlock.



Deadlock Detection and Prevention Strategies

Solution #1: Detect and Fix

- **Algorithm**: Scan the resource allocation graph and detect cycles.
- **Fix**:
  - Remove one thread and reassign its resources.
  - Rollback actions of one thread (requires checkpoints).
- **Thread Suppression**: Forcefully terminate a thread and reassign its resources (used in OSs like Unix, Windows).
- **Rollback Actions**: Restore processes to a previous safe state.

Examples

1. Detecting Cycles in the Resource Allocation Graph

We use graph theory to represent process-resource dependencies and detect cycles using DFS.

```
from collections import defaultdict

```

```
class ResourceAllocationGraph:
    def __init__(self):
        self.graph = defaultdict(list)

    def add_edge(self, u, v):
        self.graph[u].append(v)

    def _has_cycle_util(self, node, visited, rec_stack):
        visited[node] = True
        rec_stack[node] = True

        for neighbor in self.graph[node]:
            if not visited[neighbor]:
                if self._has_cycle_util(neighbor, visited, re
c_stack):
                    return True
            elif rec_stack[neighbor]:
                return True

        rec_stack[node] = False
        return False

    def has_cycle(self):
        visited = {node: False for node in self.graph}
        rec_stack = {node: False for node in self.graph}

        for node in self.graph:
            if not visited[node]:
                if self._has_cycle_util(node, visited, rec_st
ack):
                    return True
        return False

# Example Usage
rag = ResourceAllocationGraph()
rag.add_edge("P1", "R1")  # Process P1 requests R1
```

```
rag.add_edge("R1", "P2")  # R1 is assigned to P2
rag.add_edge("P2", "R2")  # Process P2 requests R2
rag.add_edge("R2", "P1")  # R2 is assigned to P1 → CYCLE!

if rag.has_cycle():
    print("Deadlock detected!")
else:
    print("No deadlock.")
```

Output:

```
nginx
CopiaModifica
Deadlock detected!
```

If the graph contains a cycle, it means a deadlock has occurred.

---

2. Terminating a Thread to Resolve Deadlock

If a deadlock is detected, we can choose to terminate one of the involved processes and release its resources.

```
import threading
import time

lock1 = threading.Lock()
lock2 = threading.Lock()

def process1():
    lock1.acquire()
    print("Process 1: acquired lock1")
    time.sleep(1)
    lock2.acquire()  # Indefinite wait → deadlock
    print("Process 1: acquired lock2")
    lock2.release()
    lock1.release()
```

```
def process2():
    lock2.acquire()
    print("Process 2: acquired lock2")
    time.sleep(1)
    lock1.acquire()  # Indefinite wait → deadlock
    print("Process 2: acquired lock1")
    lock1.release()
    lock2.release()

t1 = threading.Thread(target=process1)
t2 = threading.Thread(target=process2)

t1.start()
t2.start()

time.sleep(2)  # Detect that threads are blocked
print("Deadlock detected! Forcibly terminating Process 1.")
t1._stop()  # Unsafe method, used here for demonstration purp
oses
```

⚠️ **Note:** `_stop()` is an unsafe method and should not be used in production code. However, forcibly terminating a thread can sometimes resolve a deadlock situation.

---

3. Rolling Back Actions with Checkpoints

An alternative to termination is rolling back the process state to a previous checkpoint.

```
class Process:
    def __init__(self, name):
        self.name = name
        self.state = "Initial"
        self.checkpoint = None
```

```
    def save_checkpoint(self):
        self.checkpoint = self.state

    def rollback(self):
        if self.checkpoint:
            self.state = self.checkpoint
            print(f"{self.name} rollback performed. State res
tored to: {self.state}")
        else:
            print(f"{self.name} has no saved checkpoint.")

    def execute(self, new_state):
        self.save_checkpoint()
        self.state = new_state
        print(f"{self.name} executed, new state: {self.stat
e}")

# Example Usage
p1 = Process("P1")
p1.execute("Allocating Resource A")
p1.execute("Waiting for Resource B")  # Critical point

# Deadlock detected, rollback
print("Deadlock detected! Performing rollback...")
p1.rollback()
```

Output:

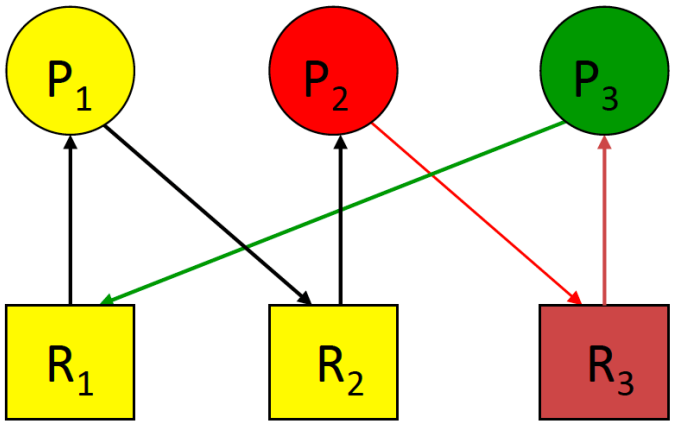
```
less
CopiaModifica
P1 executed, new state: Allocating Resource A
P1 executed, new state: Waiting for Resource B
Deadlock detected! Performing rollback...
P1 rollback performed. State restored to: Allocating Resource A
```

This approach allows a process to revert to a safe state instead of being terminated.

Solution #2: Deadlock Prevention

Eliminate one of the four necessary conditions:

- **Lock Ordering**: Always acquire locks in the same order (e.g., alphabetically).
- **Release Resources and Retry**: Avoid "wait while holding" by retrying instead of blocking.
- **Acquire All Needed Resources in Advance**: Ensures no waiting occurs while holding.
- **Resource Virtualization**: E.g., spooling for printers avoids mutual exclusion.



R3: Resource with maximum index

P_3 violates the constraint of sorted requests

P_3 causes circular waiting

```c
pthread_mutex_t lockA, lockB;

void *thread1(void *arg) {
    pthread_mutex_lock(&lockA);
    pthread_mutex_lock(&lockB);
    // Critical section
    pthread_mutex_unlock(&lockB);
```

```
    pthread_mutex_unlock(&lockA);
    return NULL;
}

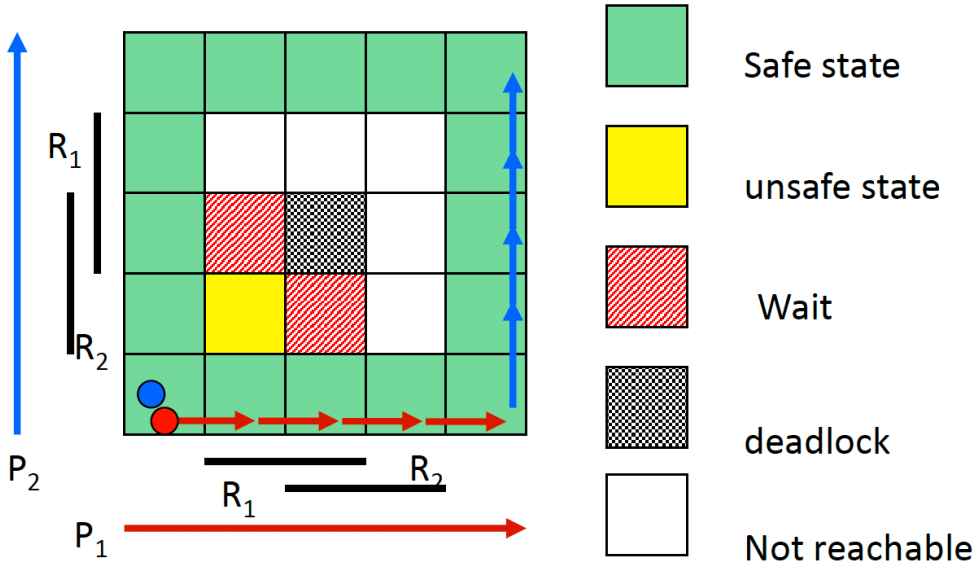
void *thread2(void *arg) {
    pthread_mutex_lock(&lockA);  // Same order as thread1
    pthread_mutex_lock(&lockB);
    // Critical section
    pthread_mutex_unlock(&lockB);
    pthread_mutex_unlock(&lockA);
    return NULL;
}
```

Solution #3: Banker's Algorithm

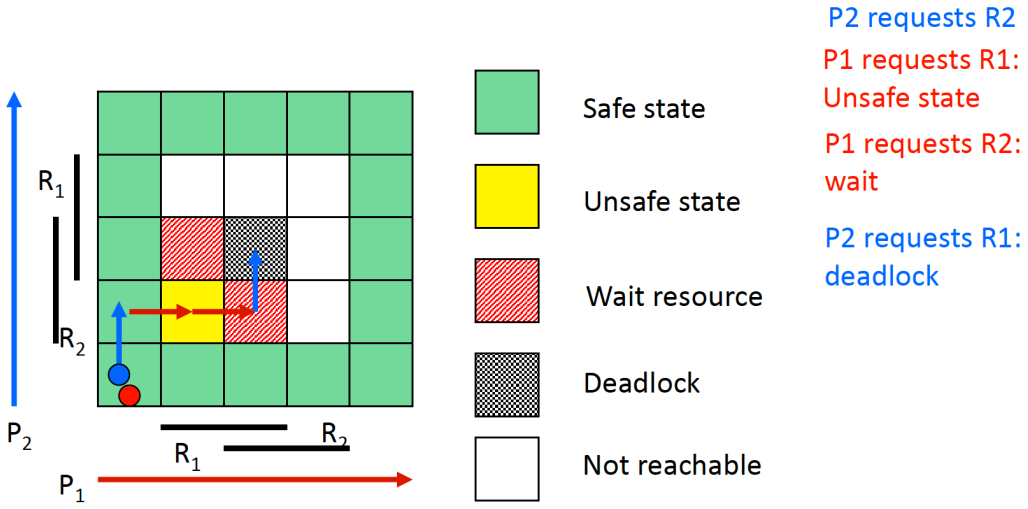
- **Predeclare maximum resource needs**
- **Allocate dynamically, ensuring safe state**
- **A request is granted only if it does not lead to an unsafe state**

Banker's Algorithm: Safe vs. Unsafe States

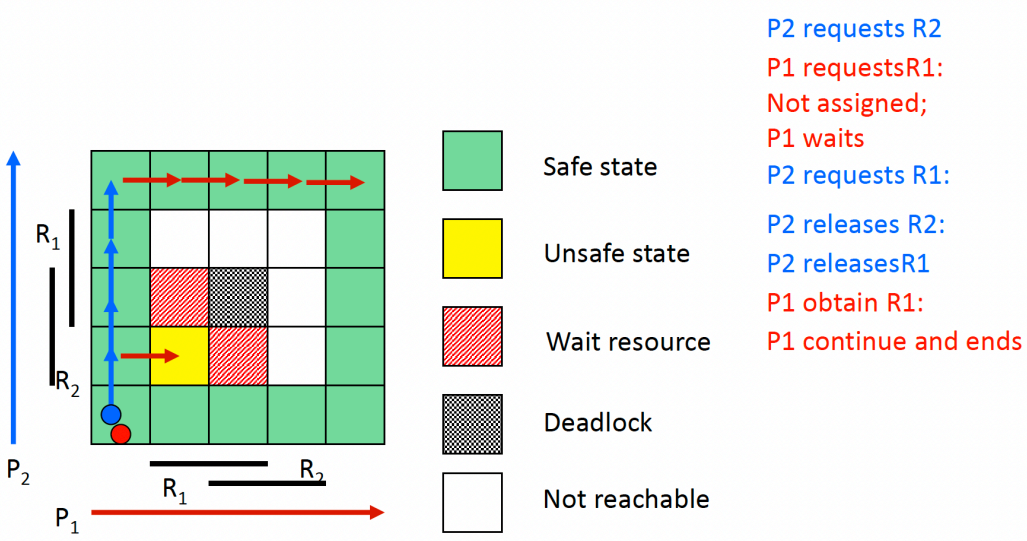
- **Safe State**: A sequence exists where all processes can complete.
    1. System that evolves in safe states



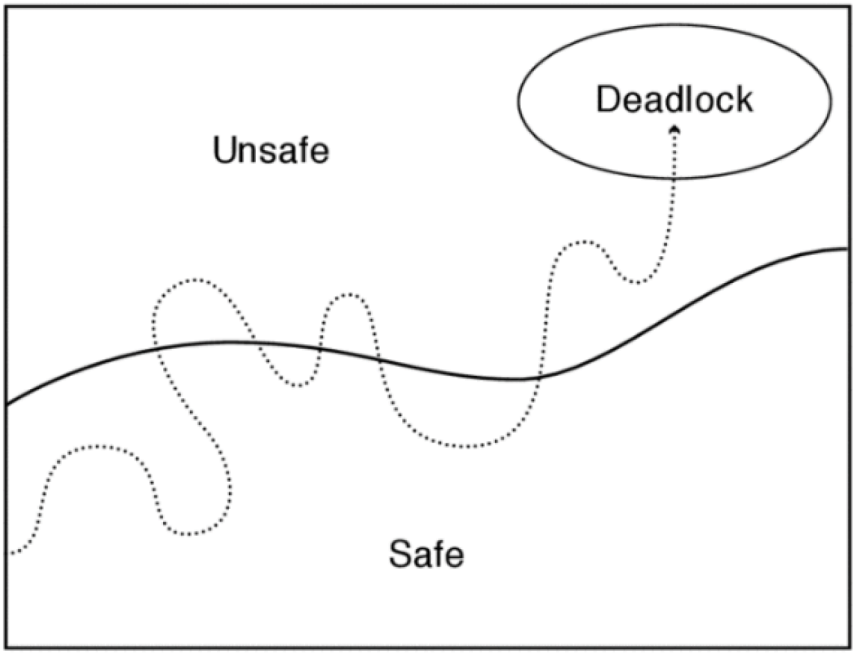
2. A system in an unsafe state can reach a deadlock



3. The banker does not accept requests that lead to unsafe states



- **Unsafe State**: Some sequence can lead to deadlock.
- **Doomed State**: Every possible computation leads to deadlock.



Banker's Algorithm Implementation

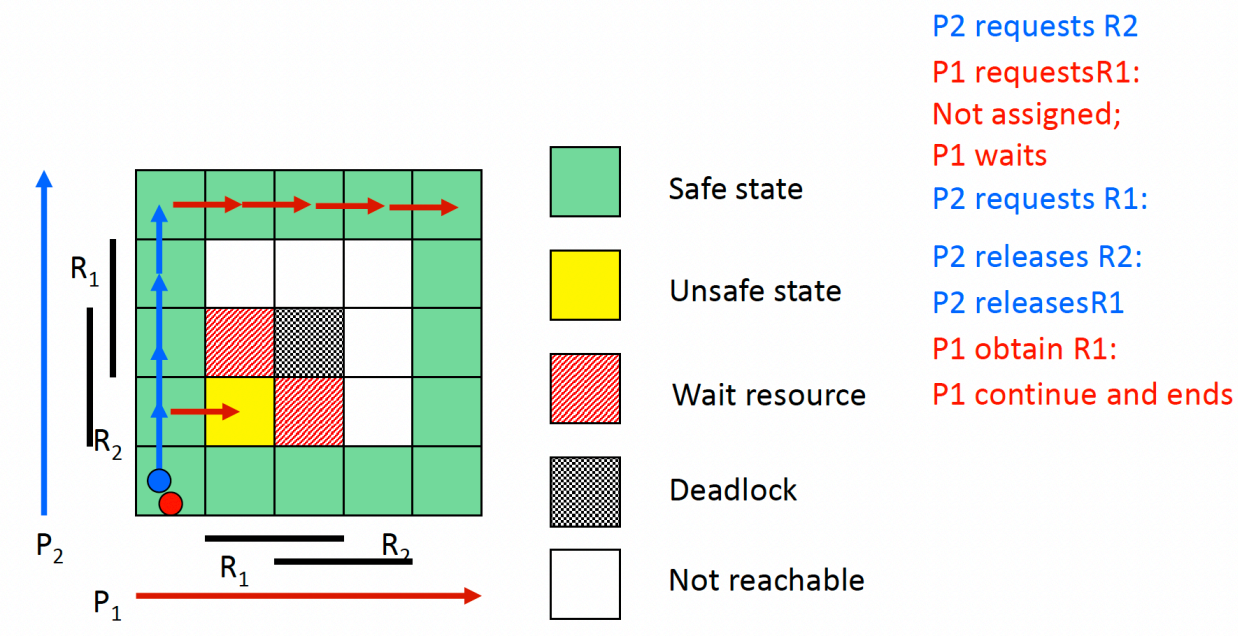
- Track available resources (`D`), allocated resources (`A`), and remaining needs (`E`)
- Process requests only if they maintain a safe state

- **Algorithm Steps:**
  1. Check if a process's request keeps the system in a safe state.
  2. If granting the request results in a safe state, allocate resources.
  3. If not, the process must wait.

Example: Simple Banker's Algorithm Execution

Process	Max Need	Allocated	Remaining Need
P1	7	3	4
P2	5	2	3
P3	8	4	4

- Suppose available resources: `D = 3`
- P2 requests 2 more resources.
- If granted, available `D` becomes `1`, making the system unsafe.
- Therefore, request is denied, and P2 must wait.



Banker's Algorithm Implementation in Python

```
import numpy as np

def is_safe_state(available, max_need, allocated):
    num_processes, num_resources = allocated.shape
    work = available.copy()
    finish = np.zeros(num_processes, dtype=bool)

    while True:
        found = False
        for i in range(num_processes):
            if not finish[i] and all(max_need[i] - allocated
[i] <= work):
                work += allocated[i]
                finish[i] = True
                found = True
        if not found:
            break

    return all(finish)

# Example Usage
available = np.array([3, 3, 2])
max_need = np.array([[7, 5, 3], [3, 2, 2], [9, 0, 2], [2, 2,
2], [4, 3, 3]])
allocated = np.array([[0, 1, 0], [2, 0, 0], [3, 0, 2], [2, 1,
1], [0, 0, 2]])

print("Safe State:", is_safe_state(available, max_need, alloc
ated))
```